实验七　SpringMVC 控制器

一、实验目的及任务

通过该实验，掌握创建基于spring boot的框架生成方法，掌握SpringMVC控制器的编写方法。能够使用 Postman 测试工具测试编写的控制器方法。

二、实验环境及条件

主机操作系统为 Win11,jdk17.

三、实验实施步骤

略

四、实验报告内容

1 用 postman 测试 RequestController.java 和 ResponseController.java 中每个控制器方法，给出你的测试数据和测试结果截图。

测试 RequestController.java：

```
//1. 简单参数
@RequestMapping("/simpleParam")
public String simpleParam(@RequestParam(name = "name", required
= false) String username, Integer age){
    System.out.println(username+ ":" + age);
    return "OK";
}
```

http://localhost:8080/simplePojo?name=Tom&age=20

Save

GET http://localhost:8080/simpleParam?name=Tom&age=20 Send

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookie

Query Params

KEY	VALUE	DESCRIPTION
name	Tom	
age	20	
Key	Value	Description

Body Cookies Headers (5) Test Results 200 OK 8 ms 165 B Save Response

Pretty Raw Preview Visualize Text

1 OK

```
//2. 实体参数
@RequestMapping("/simplePojo")
public String simplePojo(User user){
    System.out.println(user);
    return "OK";
}
```

http://localhost:8080/simplePojo?name=Tom&age=20

Save

GET http://localhost:8080/simplePojo?name=Tom&age=20 Send

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

KEY	VALUE	DESCRIPTION
name	Tom	
age	20	
Key	Value	Description

Body Cookies Headers (5) Test Results 200 OK 128 ms 165 B Save Response

Pretty Raw Preview Visualize Text

1 OK

```
//3. 数组集合参数
@RequestMapping("/arrayParam")
public String arrayParam(String[] hobby){
    System.out.println(Arrays.toString(hobby));
```

```
    return "OK";
}
```

http://localhost:8080/simplePojo?name=Tom&age=20

Save

GET http://localhost:8080/arrayParam?hobby=game&hobby=java Send

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

KEY	VALUE	DESCRIPTION	Bulk Edit
hobby	game		
hobby	java		
Key	Value	Description	

Body Cookies Headers (5) Test Results 200 OK 12 ms 165 B Save Response

Pretty Raw Preview Visualize Text

```
1 OK
```

```
//4. 日期时间参数
@RequestMapping("/dateParam")
public String dateParam(@DateTimeFormat(pattern = "yyyy-MM-dd
HH:mm:ss") LocalDateTime updateTime){
    System.out.println(updateTime);
    return "OK";
}
```

http://localhost:8080/simplePojo?name=Tom&age=20

Save

GET http://localhost:8080/dateParam?updateTime=2024-04-22 10:05:45 Send

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

KEY	VALUE	DESCRIPTION	Bulk Edit
updateTime	2024-04-22 10:05:45		
Key	Value	Description	

Body Cookies Headers (5) Test Results 200 OK 7 ms 165 B Save Response

Pretty Raw Preview Visualize Text

```
1 OK
```

```
//5. json 参数
@RequestMapping("/jsonParam")
public String jsonParam(@RequestBody User user){
```

```
    System.out.println(user);
    return "OK";
}
```

http://localhost:8080/simplePojo?name=Tom&age=20

GET http://localhost:8080/jsonParam

Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

```
1 {
2     "name":"Tom",
3     "age":10,
4     "address":{
5         "province":"beijing",
6         "city":"beijing"
7     }
8 }
```

Body Cookies Headers (5) Test Results 200 OK 50 ms 165 B Save Response

Pretty Raw Preview Visualize Text

```
1 OK
```

```
//6. 路径参数
@RequestMapping("/path/{id}")
public String pathParam(@PathVariable Integer id){
    System.out.println(id);
    return "OK";
}
```

http://localhost:8080/simplePojo?name=Tom&age=20

GET http://localhost:8080/path/1

Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies

Query Params

KEY	VALUE	DESCRIPTION	•••	Bulk Edit
Key	Value	Description		

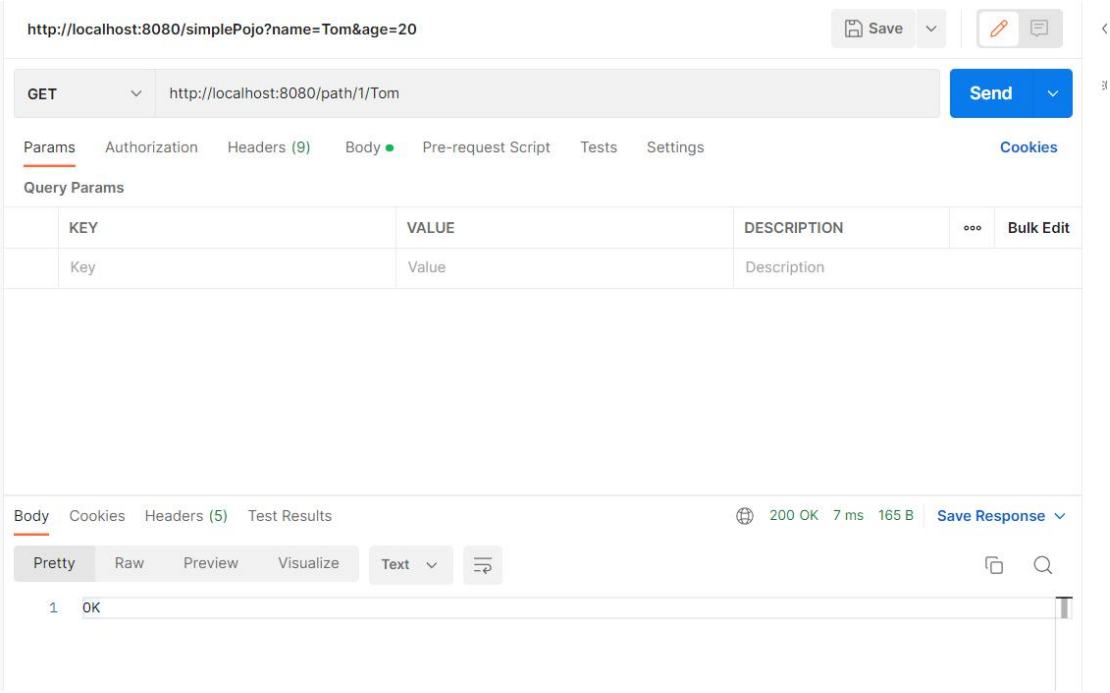
Body Cookies Headers (5) Test Results 200 OK 6 ms 165 B Save Response

Pretty Raw Preview Visualize Text

```
1 OK
```

```
@RequestMapping("/path/{id}/{name}")
public String pathParam2(@PathVariable Integer id ,
```

```
@PathVariable String name){
    System.out.println(id);
    System.out.println(name);
    return "OK";
}
```




```
@RequestMapping("/request")
public Result hello(){
    System.out.println("Hello World ~");
    //return new Result(1,"success","Hello World ~");
    return Result.success("Hello World ~");
}
```

http://localhost:8080/simplePojo?name=Tom&age=20

Save

GET http://localhost:8080/request Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies

Query Params

KEY	VALUE	DESCRIPTION	ooo	Bulk Ed
Key	Value	Description		

Body Cookies Headers (5) Test Results 200 OK 13 ms 213 B Save Response

Pretty Raw Preview Visualize JSON

```
{
    "code": 1,
    "msg": "success",
    "data": "Hello World ~"
}
```

```java
@RequestMapping("/requestGetAddr")
public Result getAddr(){
    Address addr = new Address();
    addr.setProvince("广东");
    addr.setCity("深圳");
    return Result.success(addr);
}
```

http://localhost:8080/simplePojo?name=Tom&age=20

Save

GET http://localhost:8080/requestGetAddr Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies

Query Params

KEY	VALUE	DESCRIPTION	...	Bulk Edit
Key	Value	Description		

Body Cookies Headers (5) Test Results 200 OK 5 ms 235 B Save Response

Pretty Raw Preview Visualize JSON


```
{
    "code": 1,
    "msg": "success",
    "data": {
        "province": "广东",
        "city": "深圳"
    }
}
```


```java
@RequestMapping("/requestListAddr")
public Result listAddr(){
    List<Address> list = new ArrayList<>();

    Address addr = new Address();
    addr.setProvince("广东");
    addr.setCity("深圳");

    Address addr2 = new Address();
    addr2.setProvince("陕西");
    addr2.setCity("西安");

    list.add(addr);
    list.add(addr2);
    return Result.success(list);
}
```

http://localhost:8080/simplePojo?name=Tom&age=20

Save

GET http://localhost:8080/requestListAddr Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies

Query Params

KEY	VALUE	DESCRIPTION	Bulk Edit
Key	Value	Description	

Body Cookies Headers (5) Test Results 200 OK 7 ms 275 B Save Response

Pretty Raw Preview Visualize JSON


```
{
    "code": 1,
    "msg": "success",
    "data": [
        {
            "province": "广东",
            "city": "深圳"
        },
        {
            "province": "陕西",
            "city": "西安"
        }
    ]
}
```


测试 ResponseController.java：

```java
@RequestMapping("/response")
public Result hello(){
    System.out.println("Hello World ~");
    //return new Result(1,"success","Hello World ~");
    return Result.success("Hello World ~");
}
```

http://localhost:8080/simplePojo?name=Tom&age=20

Save

GET http://localhost:8080/response Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies

Query Params

KEY	VALUE	DESCRIPTION	Bulk Edi
Key	Value	Description	

Body Cookies Headers (5) Test Results 200 OK 10 ms 213 B Save Response

Pretty Raw Preview Visualize JSON


```
{
    "code": 1,
    "msg": "success",
    "data": "Hello World ~"
}
```

```java
@RequestMapping("/responseGetAddr")
public Result getAddr(){
    Address addr = new Address();
    addr.setProvince("广东");
    addr.setCity("深圳");
    return Result.success(addr);
}
```

http://localhost:8080/simplePojo?name=Tom&age=20

Save

GET http://localhost:8080/responseGetAddr Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies

Query Params

KEY	VALUE	DESCRIPTION	...	Bulk Edit
Key	Value	Description		

Body Cookies Headers (5) Test Results 200 OK 4 ms 235 B Save Response

Pretty Raw Preview Visualize JSON

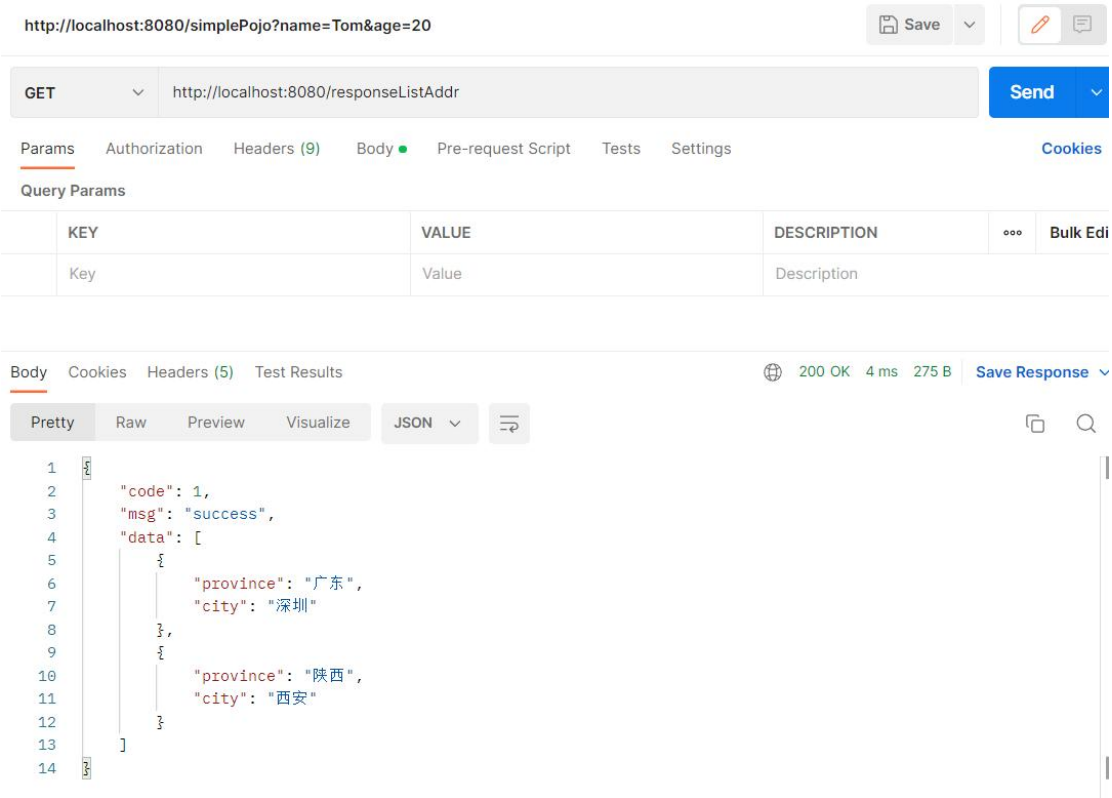
```
{
    "code": 1,
    "msg": "success",
    "data": {
        "province": "广东",
        "city": "深圳"
    }
}
```

```java
@RequestMapping("/responseListAddr")
public Result listAddr(){
    List<Address> list = new ArrayList<>();

    Address addr = new Address();
    addr.setProvince("广东");
    addr.setCity("深圳");

    Address addr2 = new Address();
    addr2.setProvince("陕西");
    addr2.setCity("西安");

    list.add(addr);
    list.add(addr2);
    return Result.success(list);
}
```



2 @Controller 和@RestController 的区别是什么？

@Controller 和@RestController 注解是用来标记一个类作为控制器的。这两个注解的主要区别在于它们处理 HTTP 请求返回数据的方式。

1. @Controller：这是 Spring 框架最初的注解，用于定义一个控制器类。当使用`@Controller`注解时，通常会返回一个视图（如 JSP 或 HTML 页面）。如果方法需要返回非 HTML 内容，比如 JSON 或 XML，那么通常需要配合`@ResponseBody`注解来使用，表示该方法的返回值应该直接作为 HTTP 响应体返回给客户端。

2. @RestController：这是一个特殊的控制器注解，它是@Controller 和@ResponseBody 注解的组合。使用@RestController 注解的类，其中的每个方法默认都会返回对象数据而不是视图。这意味着返回的数据将被自动序列化为 JSON（或其他配置的格式，如 XML），并直接用作 HTTP 响应体。

3 SpringMVC 控制器方法有哪些常用的返回值类型，分别适用什么情况？

1. String：返回视图名称，适用于需要渲染 HTML 页面的场景。返回的字符串通常会被解析为逻辑视图名称，通过视图解析器转换为实际的视图（如 JSP 或 HTML 页面）。

2. void：当方法处理完 HTTP 请求后，不返回任何数据。这通常用于 HTTP GET 请求，浏览器会重定向到另一个 URL 或者直接显示错误页面。

3. ModelAndView：同时返回模型数据和视图名称。模型数据是一个 Map，用于存储需要在视图中展示的数据。

4. Model：返回模型数据，通常与@Controller 一起使用，返回的模型数据会被添加到请求作用域中。

5. Map：返回模型数据，与 Model 类似，但是返回的是一个 Map 对象。

6. 实体对象或集合：返回 JSON 或 XML 格式的数据，通常与@RestController 或@ResponseBody 一起使用，适用于构建 RESTful

7. ResponseEntity：返回一个包含状态码、头部信息和主体的响应实体，可以用于精细控制 HTTP 响应。

8. RedirectView：用于重定向到一个不同的 URL，通常与@Controller 一起使用。

9. View：返回一个 View 对象，用于直接渲染视图，而不是通过视图名称。

4 你遇到了哪些问题，如何解决的？

在 requestcontroller 与 responsecontroller 中注解路径重复了，修改之后重新运行**解决报错**。通过该实验，掌握了**创建**基于 spring boot 的框架生成方法，掌握了 SpringMVC 控制器的编写方法。并能够使用 Postman 测试工具**测试**编写的控制器方法。